HSL-530

Data Analytics for Business & Society

Phase II Project Report

RetailHero Uplift Modeling: CCP vs CCU Evaluation

Submitted by

Anand Deep (21322003)
Aniket (21322005)
Madhav Mukund (21322018)
Param Mahabare (21322021)

29 April 2026

Abstract

This report presents the Phase II implementation and evaluation of a value-aware uplift-modeling pipeline for the RetailHero / X5 retail intervention dataset. Phase I established that campaign assignment was effectively randomized, that the average treatment effect was positive but modest, and that treatment response varied materially across customer segments. Phase II builds on those findings by comparing a conventional customer purchase-propensity benchmark (CCP) with three customer churn uplift (CCU) estimators under a common experimental framework. Features are reconstructed directly from the raw source files, models are trained on a stratified development split, and performance is assessed on a held-out validation sample using Qini, liftup, predictive maximum profit, and maximum profit uplift. The empirical results indicate that the uplift random forest provides the strongest incremental-targeting performance, whereas the CCP benchmark is most appropriately interpreted as a non-causal propensity baseline.

Contents

1 Introduction and Objective **1**

2 Data and Verified Phase I Findings **1**
2.1 Dataset . . . 1
2.2 Phase I Findings Revalidated from Raw Data . . . 1
2.3 Implications for Phase II Design . . . 2

3 Phase II Methodology **2**
3.1 Design Principles . . . 2
3.2 Feature Engineering . . . 2
3.3 Data Split . . . 3
3.4 Models . . . 3
3.4.1 Model A: CCP baseline . . . 3
3.4.2 Model B1: Two-model logistic uplift . . . 3
3.4.3 Model B2: Lo-style interaction logistic uplift . . . 3
3.4.4 Model B3: Uplift random forest . . . 4
3.5 Evaluation Framework . . . 4

4 Implementation Summary **4**
4.1 Code Organization . . . 4
4.2 Implementation Integrity Checks . . . 4

5 Results **5**
5.1 Selected Hyperparameters . . . 5
5.2 Validation Summary . . . 5
5.3 Interpretation . . . 5
5.4 Figures . . . 6

6 Methodological Assessment **8**
6.1 Why the pipeline is methodologically sound . . . 8
6.2 Why the model comparison is fair . . . 8
6.3 Why the conclusions are credible . . . 8

7 Limitations **9**

8 Conclusion **9**

A Relevant Code Listings **9**
A.1 Client and Product Feature Construction . . . 9
A.2 Uplift Model Wrappers . . . 13
A.3 Pipeline Orchestration . . . 16

B Reproduction Commands **19**

1 Introduction and Objective

The objective of Phase II was to construct a reproducible and methodologically coherent comparison between:

- **Model A: CCP baseline**, a standard purchase-propensity model trained only on the control group.
- **Model B: CCU models**, uplift estimators that rank customers by expected incremental response to treatment.

The central research question addressed in this phase is as follows:

> Can the intervention be targeted more efficiently by ranking customers on expected incremental value rather than raw purchase likelihood?

This distinction is important because a customer with a high unconditional probability of purchase may still be an inefficient target for campaign expenditure if that customer would have purchased in the absence of treatment. The practical value of uplift modeling therefore depends on its ability to alter the targeting order in a measurable and economically meaningful way.

2 Data and Verified Phase I Findings

2.1 Dataset

The project uses the RetailHero uplift dataset with the following raw inputs:

- `clients.csv`
- `products.csv`
- `purchases.csv`
- `uplift_train.csv`
- `uplift_test.csv`
- `uplift_sample_submission.csv`

The purchases file is the dominant data asset and contains approximately 4.4 GB of transaction history. Because of its size, the implementation uses DuckDB-backed SQL aggregation rather than loading the entire file into a single in-memory pandas frame.

2.2 Phase I Findings Revalidated from Raw Data

Phase I findings were rechecked directly from raw data via the verification script rather than relying on notebook narrative alone. The key findings are summarized in Table 1.

Table 1: Verified Phase I findings used to motivate Phase II

Finding	**Verified value**
Treatment / control ratio	0.999230 (99,981 treated / 100,058 control)
Baseline uplift	3.323 percentage points
Average uplift in low-frequency deciles 0–3	4.452 percentage points
Average uplift in high-frequency deciles 6–9	1.755 percentage points
Sure Things share	30.176% of training clients
Age anomalies	min = -7,491, max = 1,901, flagged = 1,145
Distinct non-null `level_1` categories	3
Test-set clients with purchase history	200,123 / 200,123

2.3 Implications for Phase II Design

These findings motivate the Phase II design in three ways:

1. The near-perfect treatment/control balance means uplift estimates can be interpreted causally without heavy post-hoc adjustment.
2. The modest average treatment effect means a single global response model is likely to be wasteful because the mean hides heterogeneity.
3. The concentration of uplift among lower-frequency customers means targeting by purchase probability alone is not aligned with incremental value.

3 Phase II Methodology

3.1 Design Principles

Phase II was built around four design principles:

1. **Raw-data-first reproducibility**: all Phase II features are rebuilt from `data/*.csv` rather than notebook-only exports.
2. **Comparable feature space**: all models consume the same deterministic numeric feature matrix.
3. **Value-aware evaluation**: uplift models are not judged only by response prediction, but by incremental targeting value.
4. **Transparent implementation**: the pipeline is organized into feature, model, metric, and orchestration modules rather than one monolithic script.

3.2 Feature Engineering

The final Phase II matrix contains 44 numeric features with zero missing values after deterministic filling and explicit missingness indicators. The feature groups are:

- **Demographics**: `age`, `age_clean`, `age_flagged`, gender indicators, `gender_known`
- **Redemption behavior**: redeemed flag, time-to-redeem, card tenure, and redemption recency

- **Purchase behavior**: spend, transactions, trips, items, basket metrics, stores, products, points, recency, tenure
- **Product behavior**: alcohol spend share, own-trademark spend share, netto aggregates, spend-share across the three `level_1` groups, and brand concentration (HHI)

This feature design follows directly from the empirical lessons of Phase I. Because the hashed category hierarchy is not semantically interpretable, the implementation does not impose artificial business meaning on those fields. Instead, product metadata is converted into client-level behavioral aggregates that preserve signal while remaining methodologically honest.

3.3 Data Split

The Phase II pipeline uses:

- an outer holdout split of 20% of `uplift_train` for validation,
- stratification on the joint label `(treatment_flg, target)`,
- a fixed random seed of 123,
- a smaller inner tuning split inside the training fold only.

This split design preserves an untouched validation sample while keeping the modeling pipeline sufficiently transparent to inspect, reproduce, and extend.

3.4 Models

3.4.1 Model A: CCP baseline

The CCP baseline is an `XGBClassifier` trained on control-group rows only. It predicts purchase propensity rather than treatment effect. This specification provides a strong conventional benchmark, avoids learning treatment-induced behavior when estimating organic demand, and reflects the type of model that might plausibly be deployed if uplift were ignored.

3.4.2 Model B1: Two-model logistic uplift

Two separate regularized logistic models are trained, one on treated customers and one on control customers. The uplift score is:

$$\hat{u}(x) = \hat{P}(Y = 1 \mid T = 1, X = x) - \hat{P}(Y = 1 \mid T = 0, X = x)$$

This specification is the most direct implementation of the roadmap formula and serves as a transparent linear uplift baseline.

3.4.3 Model B2: Lo-style interaction logistic uplift

A single logistic model is trained on all rows with:

- the treatment indicator as a feature,
- first-order interactions between treatment and all covariates.

The uplift score is obtained by scoring each customer twice, once under treatment and once under control, then taking the difference.

This specification provides a structured single-model alternative to the two-model approach and is useful for testing whether treatment interactions can be captured parsimoniously within one regularized estimator.

3.4.4 Model B3: Uplift random forest

The non-linear uplift model is implemented using `causalml`'s `UpliftRandomForestClassifier` with:

- `KL` split evaluation,
- string-encoded treatment labels required by the package API,
- light tuning over tree count, feature count, depth, leaf size, and regularization.

This model is the most expressive estimator in the comparison and is capable of representing non-linear treatment heterogeneity that the linear alternatives cannot capture.

3.5 Evaluation Framework

Phase II evaluates the models with four complementary views:

- **Qini curves**: how well the model ranks incremental responders
- **Liftup curves**: uplift-specific analogues of lift behavior
- **Predictive maximum profit (MP)**: used for the CCP benchmark
- **Maximum profit uplift (MPU)**: used for incremental targeting

The default economics are:

- average basket value = empirical average trip value = 4425.003 RUB
- SMS cost = 1 RUB

Sensitivity analysis then varies both assumptions.

4 Implementation Summary

4.1 Code Organization

The Phase II codebase is modular:

- `phase2_features.py`: raw-data feature construction and cache writing
- `phase2_models.py`: model wrappers, tuning, and scoring
- `phase2_metrics.py`: ranking, Qini, liftup, MP, and MPU calculations
- `phase2_pipeline.py`: full orchestration and artifact generation
- `verify_phase2_outputs.py`: artifact and consistency checks

4.2 Implementation Integrity Checks

The completed pipeline passed all current verification checks:

- 200,039 Phase II train rows and 200,123 Phase II test rows
- 44 model features
- zero missing feature values in both train and test matrices

- valid category-share algebra with maximum absolute error 1.343×10^{-14}
- finite validation and test scores for all four models
- valid target-share bounds for Qini, liftup, and sensitivity outputs

These checks are important because a submission-ready analysis should demonstrate not only competitive model performance, but also internal consistency of the underlying data and artifact pipeline.

5 Results

5.1 Selected Hyperparameters

The final selected hyperparameters from the current full run are shown in Table 2.

Table 2: Selected model hyperparameters

Model	Selected parameters
Model A - CCP baseline	`n_estimators=250`, `max_depth=3`, `learning_rate=0.05`, `subsample=0.85`, `colsample_bytree=0.85`
Model B1 - Two-model logistic	`C=1.0`
Model B2 - Lo interaction logistic	`C=10.0`
Model B3 - Uplift random forest	`n_estimators=25`, `max_features=12`, `max_depth=5`, `min_samples_leaf=100`, `min_samples_treatment=50`, `n_reg=10`

5.2 Validation Summary

The full validation summary is shown in Table 3. The validation holdout contains 40,008 rows.

Table 3: Phase II validation summary

Model	Qini AUC	Top-decile liftup	MPU	Optimal target share
Model B3 - CCU Uplift Random Forest	0.004759	3.637315	158.307694	0.7475
Model B1 - CCU Two-Model Logistic	0.003297	3.239985	146.083912	1.0000
Model B2 - CCU Lo Interaction Logistic	0.003245	3.231146	146.083912	1.0000
Model A - CCP Baseline	-0.002239	0.016284	146.083912	1.0000

For completeness, the CCP baseline also achieved a predictive maximum profit of 2742.063982 under the default economics. That figure is intentionally reported separately from MPU because it is not the same business question. Predictive MP rewards finding likely buyers. MPU rewards finding customers whose behavior is changed by treatment. Those are related but not interchangeable objectives.

5.3 Interpretation

Three observations are particularly important:

1. The uplift random forest is the strongest CCU model on both Qini and MPU.
2. The logistic uplift models show positive uplift discrimination and materially outperform the CCP baseline on top-decile liftup.
3. The CCP baseline is excellent at ranking likely buyers, but poor at isolating persuadable buyers near the top of the list.

This pattern is consistent with the evidence reported in Phase I. High-frequency customers often behave as "sure things" and can therefore dominate a purchase-propensity ranking while contributing relatively little incremental value. The uplift models correct for this by ranking customers according to estimated treatment effect rather than unconditional purchase likelihood.

5.4 Figures

Figure 1 shows the Qini curves. The uplift random forest dominates the uplift-specific ranking metric, while the CCP baseline is negative relative to random on this criterion.

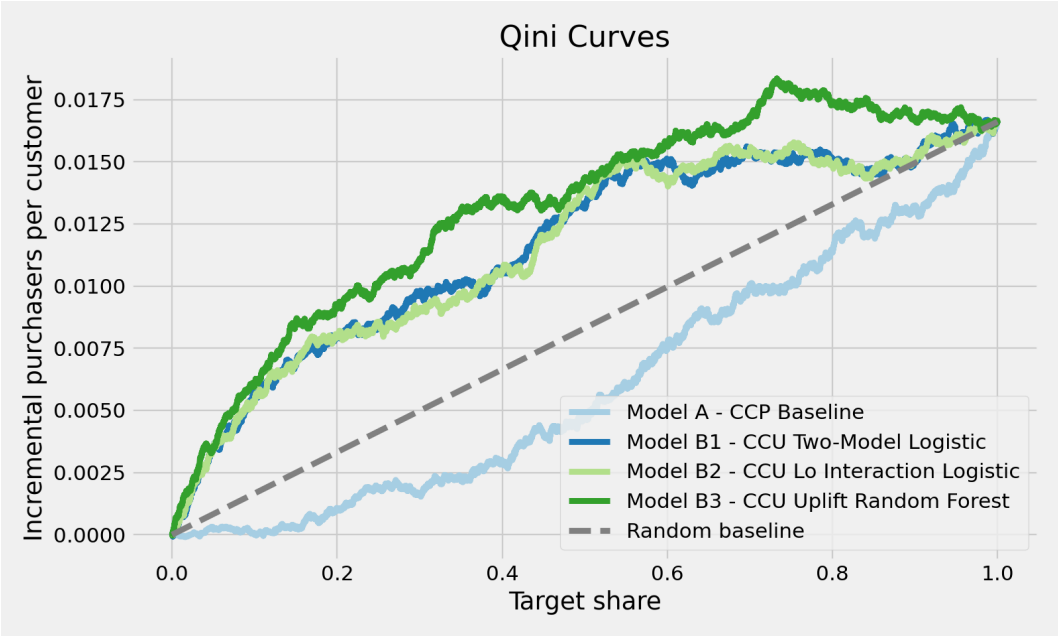

Figure 1: Qini curves on the Phase II validation holdout.

Figure 2 shows the liftup curves. The CCP baseline has very high top-decile response lift but almost no top-decile liftup, which is precisely the failure mode Phase I warned about.

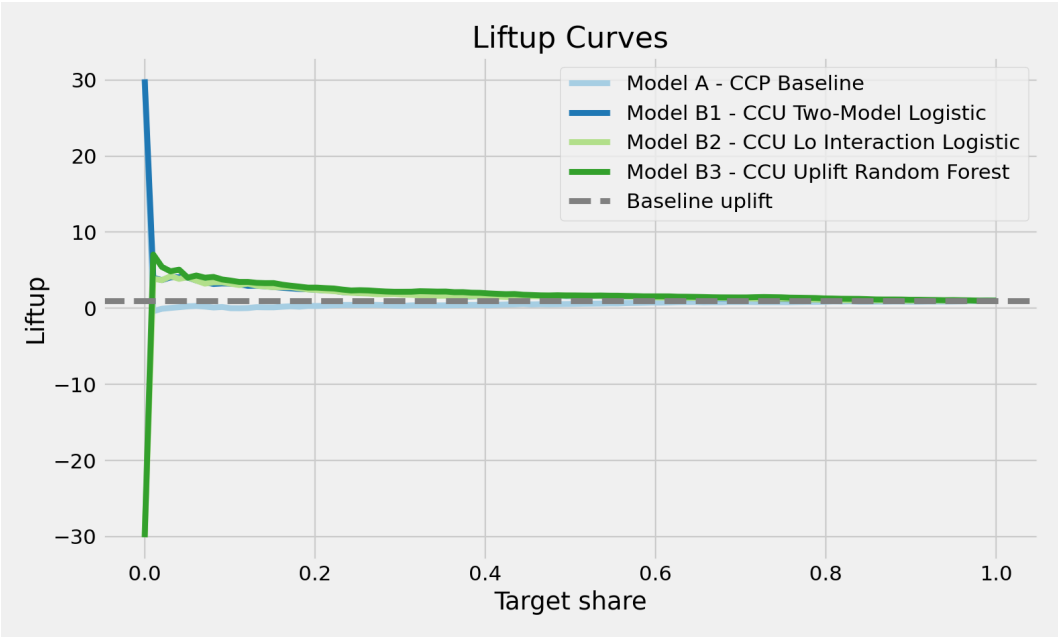

Figure 2: Liftup curves on the Phase II validation holdout.

Figure 3 compares incremental profit curves and the CCP predictive MP curve. The uplift random forest is the only model that clearly pushes the incremental-profit frontier upward.

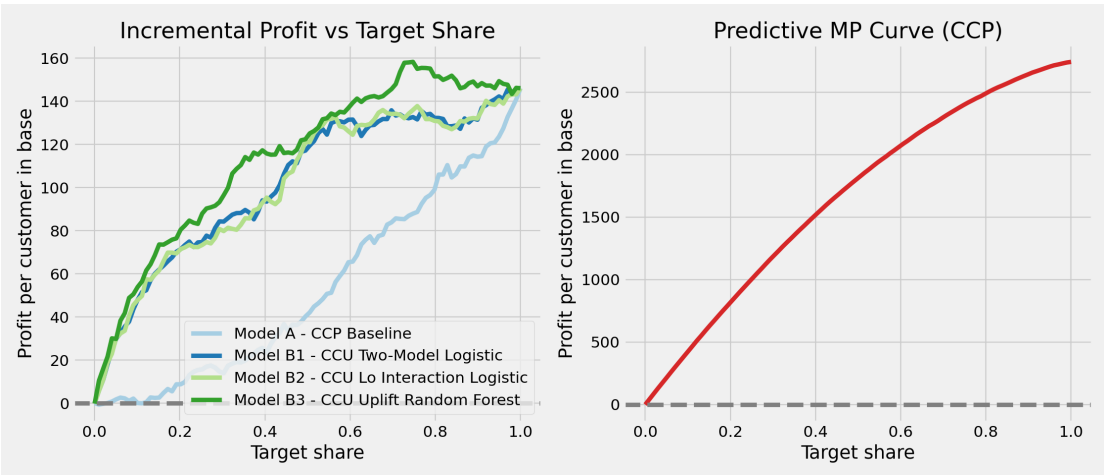

Figure 3: Incremental profit and CCP predictive MP curves.

Figure 4 shows that the relative ordering is not a single-point artifact of the chosen economics; the uplift random forest remains competitive under the current sensitivity sweep.

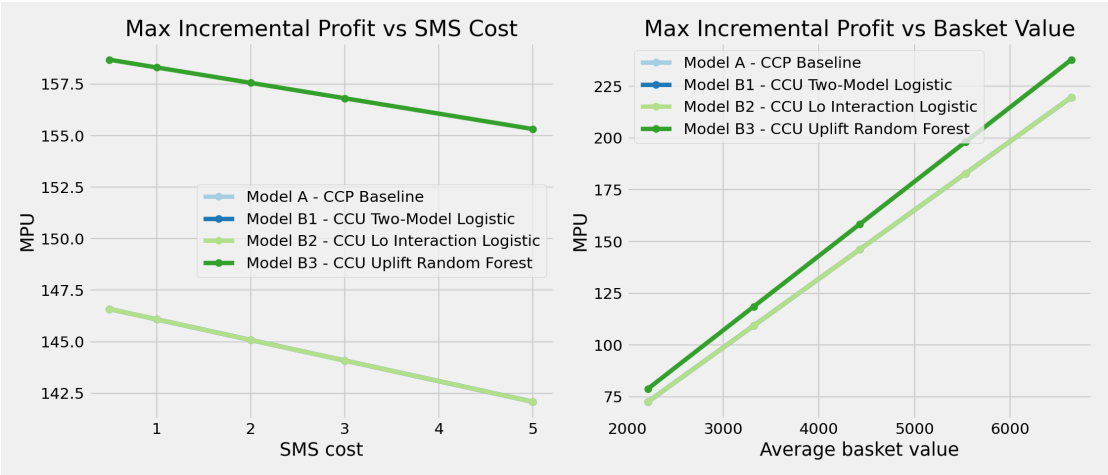

Figure 4: Sensitivity of maximum incremental profit to basket value and SMS cost assumptions.

6 Methodological Assessment

6.1 Why the pipeline is methodologically sound

The methodological soundness of the pipeline rests on the fact that it avoids three common errors:

1. **It does not confuse purchase likelihood with incremental value.**
2. **It does not rely on notebook-only, hand-built outputs.**
3. **It does not treat anonymized product categories as if they were interpretable business taxonomies.**

Instead, the implementation follows the empirical evidence established in Phase I and applies an evaluation framework that is aligned with the actual targeting objective.

6.2 Why the model comparison is fair

The comparison may be regarded as fair for four reasons:

- all models use the same raw source data,
- all models use the same engineered feature base,
- all uplift models are tuned on inner splits only,
- all final metrics come from the same outer validation set.

6.3 Why the conclusions are credible

The substantive conclusions are credible because they are supported simultaneously by:

- Phase I causal balance checks,
- full-run validation outputs,
- uplift-specific ranking curves,
- profit-oriented summaries,
- formal verification scripts for both feature integrity and saved artifacts.

7 Limitations

Several limitations remain:

- The hashed product hierarchy limits semantic interpretation.
- The current profit analysis depends on default economic assumptions, though sensitivity analysis reduces that risk.
- The uplift random forest uses a heavier third-party package and is slower than the logistic baselines.
- The implementation is an experimental analysis pipeline, not a production scoring service.

These limitations are acceptable at the present stage because the purpose of Phase II was comparative methodology and defensible empirical analysis rather than production deployment.

8 Conclusion

Phase II successfully translated the Phase I findings into a reproducible uplift-modeling comparison. The resulting pipeline demonstrates that:

- a conventional CCP purchase-propensity ranking is not sufficient for incremental targeting,
- uplift-specific models are empirically justified on this dataset,
- the uplift random forest is currently the strongest model under the implemented evaluation framework.

Taken together, the results support the central claim of the project: customer targeting based on expected incremental response is more defensible than targeting based solely on unconditional purchase probability.

A Relevant Code Listings

The appendix includes the most relevant implementation excerpts directly from the working codebase. Inline comments have been added to the source files to clarify the non-obvious modeling and data-engineering decisions referenced in the main text.

A.1 Client and Product Feature Construction

```python
def _read_clients(clients_path: str | Path, campaign_date: pd.Timestamp) -> pd.DataFrame:
    clients = pd.read_csv(
        clients_path,
        parse_dates=["first_issue_date", "first_redeem_date"],
    )

    # Preserve the original age field but add a bounded version for modeling and
    # an explicit anomaly flag so bad source values are still auditable.
    clients["age_flagged"] = ((clients["age"] < 0) | (clients["age"] > 100)).astype(int)
    clients["age_clean"] = clients["age"].clip(lower=0, upper=100)

    # Expand the anonymized gender code into a model-friendly indicator set.
    clients["gender_M"] = (clients["gender"] == "M").astype(int)
```

```python
    clients["gender_F"] = (clients["gender"] == "F").astype(int)
    clients["gender_U"] = (clients["gender"] == "U").astype(int)
    clients["gender_known"] = (clients["gender"] != "U").astype(int)

    # Redemption timing is structurally missing for customers who never redeemed,
    # so we pair every filled value with a missingness indicator.
    clients["ever_redeemed"] = clients["first_redeem_date"].notna().astype(int)
    clients["days_to_redeem"] = (
        clients["first_redeem_date"] - clients["first_issue_date"]
    ).dt.days
    clients["days_to_redeem_missing"] = clients["days_to_redeem"].isna().astype(int)
    clients["days_to_redeem"] = clients["days_to_redeem"].fillna(-1)

    clients["issue_age_days"] = (campaign_date - clients["first_issue_date"]).dt.days
    clients["redeem_recency_days"] = (
        campaign_date - clients["first_redeem_date"]
    ).dt.days
    clients["redeem_recency_missing"] = clients["redeem_recency_days"].isna().astype(int)
    clients["redeem_recency_days"] = clients["redeem_recency_days"].fillna(-1)

    return clients[
        [
            "client_id",
            "age",
            "age_clean",
            "age_flagged",
            "gender_M",
            "gender_F",
            "gender_U",
            "gender_known",
            "ever_redeemed",
            "days_to_redeem",
            "days_to_redeem_missing",
            "issue_age_days",
            "redeem_recency_days",
            "redeem_recency_missing",
        ]
    ].copy()
```

Listing 1: Annotated client-level feature construction in `phase2_features.py`

```python
def _build_product_behavior_features(
    purchases_path: str | Path,
    products_path: str | Path,
    product_meta: dict[str, Any],
) -> pd.DataFrame:
    netto_p99 = product_meta["netto_p99"]
    level1_categories = product_meta["level1_categories"]

    # Build one spend accumulator per observed hashed top-level category so the
    # final client-level matrix captures category mix without assuming semantics.
    category_spend_cols = ",\n".join(
        [
            f"    sum(case when level_1 = '{category}' then purchase_sum else 0 end) "
            f"as level1_spend_{category}"
            for category in level1_categories
        ]
```

```
    )

    conn = _connect()
    try:
        # Push the wide product-behavior aggregation into DuckDB so the largest
        # table never has to be materialized in pandas at full granularity.
        query = f"""
        with products_clean as (
          select
            product_id,
            level_1,
            brand_id,
            coalesce(is_alcohol, 0) as is_alcohol,
            coalesce(is_own_trademark, 0) as is_own_trademark,
            case
              when netto is null then null
              else least(netto, {netto_p99})
            end as netto_clean
          from read_csv(?, header=true, columns={PRODUCT_COLUMNS_SQL})
        ),
        joined as (
          select
            purchases.client_id,
            coalesce(purchases.purchase_sum, 0) as purchase_sum,
            coalesce(purchases.product_quantity, 0) as product_quantity,
            products_clean.level_1,
            products_clean.brand_id,
            products_clean.is_alcohol,
            products_clean.is_own_trademark,
            products_clean.netto_clean
          from read_csv(?, header=true, columns={PURCHASE_COLUMNS_SQL}) purchases
          left join products_clean
          using (product_id)
        ),
        client_rollup as (
          select
            client_id,
            sum(purchase_sum) as product_join_total_spend,
            sum(case when is_alcohol = 1 then purchase_sum else 0 end) as alcohol_spend,
            sum(case when is_own_trademark = 1 then purchase_sum else 0 end) as own_trademark_spend,
            sum(case when level_1 is null then purchase_sum else 0 end) as unknown_level1_spend,
            count(distinct brand_id) as n_brands,
            sum(coalesce(netto_clean, 0) * product_quantity) as netto_total,
            sum(case when netto_clean is null then product_quantity else 0 end) as missing_netto_quantity,
{category_spend_cols}
          from joined
          group by client_id
        ),
        brand_spend as (
          select
            client_id,
            brand_id,
            sum(purchase_sum) as brand_spend
          from joined
          where brand_id is not null
          group by client_id, brand_id
        ),
        brand_hhi as (
```

```
          select
            brand_spend.client_id,
            sum(power(brand_spend.brand_spend / nullif(client_rollup.product_join_total_spend, 0), 2)) as brand_hhi
          from brand_spend
          inner join client_rollup
            on brand_spend.client_id = client_rollup.client_id
          group by brand_spend.client_id
        )
        select
          client_rollup.*,
          coalesce(brand_hhi.brand_hhi, 0) as brand_hhi
        from client_rollup
        left join brand_hhi
          on client_rollup.client_id = brand_hhi.client_id
        order by client_rollup.client_id
        """
        features = conn.execute(
            query,
            [
                _normalize_path(products_path),
                _normalize_path(purchases_path),
            ],
        ).df()
    finally:
        conn.close()

    return features


def _add_share_features(df: pd.DataFrame, level1_categories: list[str]) -> pd.DataFrame:
    # Shares normalize raw spend totals into comparable behavioral signals.
    total_spend = df["total_spend"].replace(0, np.nan)

    df["alcohol_spend_share"] = (df["alcohol_spend"] / total_spend).fillna(0)
    df["own_trademark_spend_share"] = (df["own_trademark_spend"] / total_spend).fillna(0)
    df["unknown_level1_spend_share"] = (df["unknown_level1_spend"] / total_spend).fillna(0)

    for category in level1_categories:
        spend_col = f"level1_spend_{category}"
        share_col = f"level1_share_{category}"
        df[share_col] = (df[spend_col] / total_spend).fillna(0)

    item_denom = df["n_items"].replace(0, np.nan)
    df["avg_netto_per_item"] = (df["netto_total"] / item_denom).fillna(0)
    df["missing_netto_share"] = (df["missing_netto_quantity"] / item_denom).fillna(0)
    return df


def _fill_missing_with_indicators(
    train: pd.DataFrame,
    test: pd.DataFrame,
    id_columns: set[str],
) -> tuple[pd.DataFrame, pd.DataFrame, list[str]]:
    # All downstream models expect a fully numeric matrix. Missingness indicators
    # preserve information while deterministic sentinel filling removes NaNs.
    combined = pd.concat(
        [
```

```
            train.assign(_dataset="train"),
            test.assign(_dataset="test"),
        ],
        ignore_index=True,
        sort=False,
    )

    feature_columns = [
        column
        for column in combined.columns
        if column not in id_columns and column != "_dataset"
    ]

    for column in feature_columns:
        if combined[column].isna().any():
            indicator_col = f"{column}_missing"
            combined[indicator_col] = combined[column].isna().astype(int)

            observed = combined[column].dropna()
            if observed.empty:
                fill_value = -1.0
            elif (observed >= 0).all():
                # Use a shared negative sentinel for naturally non-negative fields.
                fill_value = -1.0
            else:
                fill_value = float(observed.min()) - 1.0

            combined[column] = combined[column].fillna(fill_value)
        combined[column] = pd.to_numeric(combined[column], errors="raise")

    train_out = combined.loc[combined["_dataset"] == "train"].drop(columns="_dataset").reset_index(drop=True)
    test_out = combined.loc[combined["_dataset"] == "test"].drop(columns="_dataset").reset_index(drop=True)

    feature_columns = [
        column
        for column in train_out.columns
        if column not in id_columns
    ]
    return train_out, test_out, feature_columns
```

Listing 2: Annotated product-behavior aggregation and share-feature construction in `phase2_features.py`

A.2 Uplift Model Wrappers

```
    def __init__(self, C: float, random_state: int, max_iter: int = 1000) -> None:
        self.C = C
        self.random_state = random_state
        self.max_iter = max_iter
        self.treatment_model = Pipeline(
            [
                ("scale", StandardScaler()),
                (
                    "logit",
                    LogisticRegression(
                        C=self.C,
                        max_iter=self.max_iter,
                        solver="lbfgs",
```

```
                        random_state=self.random_state,
                    ),
                ),
            ]
        )
        self.control_model = Pipeline(
            [
                ("scale", StandardScaler()),
                (
                    "logit",
                    LogisticRegression(
                        C=self.C,
                        max_iter=self.max_iter,
                        solver="lbfgs",
                        random_state=self.random_state,
                    ),
                ),
            ]
        )

    def fit(self, X: np.ndarray, treatment: np.ndarray, y: np.ndarray) -> "TwoModelLogisticUplift":
        # Fit the treated and control response surfaces independently, then
        # compare their predicted probabilities at scoring time.
        self.treatment_model.fit(X[treatment == 1], y[treatment == 1])
        self.control_model.fit(X[treatment == 0], y[treatment == 0])
        return self

    def predict_uplift(self, X: np.ndarray) -> np.ndarray:
        treated_scores = self.treatment_model.predict_proba(X)[:, 1]
        control_scores = self.control_model.predict_proba(X)[:, 1]
        return treated_scores - control_scores


class LoInteractionLogisticUplift:
    def __init__(self, C: float, random_state: int, max_iter: int = 1000) -> None:
        self.C = C
        self.random_state = random_state
        self.max_iter = max_iter
        self.model = Pipeline(
            [
                ("scale", StandardScaler()),
                (
                    "logit",
                    LogisticRegression(
                        C=self.C,
                        max_iter=self.max_iter,
                        solver="lbfgs",
                        random_state=self.random_state,
                    ),
                ),
            ]
        )

    @staticmethod
    def _design_matrix(X: np.ndarray, treatment: np.ndarray) -> np.ndarray:
        # Lo-style uplift uses one shared linear model with explicit treatment
        # interactions rather than two separate conditional models.
        treatment = treatment.reshape(-1, 1).astype(float)
        return np.hstack([X, treatment, X * treatment])

```

```python
    def fit(self, X: np.ndarray, treatment: np.ndarray, y: np.ndarray) -> "LoInteractionLogisticUplift":
        self.model.fit(self._design_matrix(X, treatment), y)
        return self

    def predict_counterfactual(self, X: np.ndarray, treatment_value: int) -> np.ndarray:
        treatment = np.full(X.shape[0], treatment_value, dtype=int)
        return self.model.predict_proba(self._design_matrix(X, treatment))[:, 1]

    def predict_uplift(self, X: np.ndarray) -> np.ndarray:
        return self.predict_counterfactual(X, 1) - self.predict_counterfactual(X, 0)


class ControlOnlyXGBModel:
    def __init__(self, random_state: int, **params: Any) -> None:
        self.random_state = random_state
        self.params = params
        self.model = XGBClassifier(
            objective="binary:logistic",
            eval_metric="logloss",
            random_state=self.random_state,
            n_jobs=-1,
            tree_method="hist",
            verbosity=0,
            **self.params,
        )

    def fit(self, X: np.ndarray, treatment: np.ndarray, y: np.ndarray) -> "ControlOnlyXGBModel":
        # Estimate the organic purchase propensity from untreated behavior only.
        control_mask = treatment == 0
        self.model.fit(X[control_mask], y[control_mask])
        return self

    def predict_score(self, X: np.ndarray) -> np.ndarray:
        return self.model.predict_proba(X)[:, 1]


class CausalMLUpliftRandomForest:
    def __init__(self, random_state: int, **params: Any) -> None:
        self.random_state = random_state
        self.params = params
        self.model = UpliftRandomForestClassifier(
            control_name="control",
            random_state=self.random_state,
            n_jobs=-1,
            evaluationFunction="KL",
            **self.params,
        )

    @staticmethod
    def _encode_treatment(treatment: np.ndarray) -> np.ndarray:
        # causalml expects string labels for the control/treatment arms.
        return np.where(treatment == 1, "treatment", "control")

    def fit(self, X: np.ndarray, treatment: np.ndarray, y: np.ndarray) -> "CausalMLUpliftRandomForest":
        self.model.fit(
            X.astype(np.float32),
```

```
            self._encode_treatment(treatment),
            y.astype(int),
        )
        return self

    def predict_uplift(self, X: np.ndarray) -> np.ndarray:
        return self.model.predict(X.astype(np.float32)).reshape(-1)
```

Listing 3: Annotated uplift model wrappers in `phase2_models.py`

A.3 Pipeline Orchestration

```
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Phase II CCP vs CCU pipeline.")
    parser.add_argument("--data-dir", type=Path, default=Path(__file__).resolve().parent / "data")
    parser.add_argument("--output-dir", type=Path, default=Path(__file__).resolve().parent / "outputs" / "phase2")
    parser.add_argument("--random-seed", type=int, default=123)
    parser.add_argument("--sms-cost", type=float, default=1.0)
    parser.add_argument("--avg-basket-value", type=float, default=None)
    parser.add_argument("--reuse-features", action="store_true")
    parser.add_argument("--skip-sensitivity-sweep", action="store_true")
    return parser.parse_args()


def main() -> None:
    args = parse_args()
    args.output_dir.mkdir(parents=True, exist_ok=True)

    # Rebuild from raw CSVs unless the caller explicitly opts into cached features.
    bundle = build_phase2_feature_bundle(
        data_dir=args.data_dir,
        output_dir=args.output_dir,
        campaign_date=CAMPAIGN_DATE,
        reuse_cache=args.reuse_features,
    )
    feature_checks = validate_feature_bundle(bundle)

    avg_basket_value = (
        args.avg_basket_value
        if args.avg_basket_value is not None
        else bundle.metadata["avg_basket_value_empirical"]
    )

    outer_train, outer_valid = split_train_validation(
        bundle.train,
        seed=args.random_seed,
        validation_size=0.20,
    )

    model_results = [
        train_ccp_xgboost(
            outer_train,
            outer_valid,
            bundle.test,
            bundle.feature_columns,
            args.random_seed,
            avg_basket_value,
            args.sms_cost,
```

```
        ),
        train_two_model_logistic(
            outer_train,
            outer_valid,
            bundle.test,
            bundle.feature_columns,
            args.random_seed,
            avg_basket_value,
            args.sms_cost,
        ),
        train_lo_logistic(
            outer_train,
            outer_valid,
            bundle.test,
            bundle.feature_columns,
            args.random_seed,
            avg_basket_value,
            args.sms_cost,
        ),
        train_uplift_random_forest(
            outer_train,
            outer_valid,
            bundle.test,
            bundle.feature_columns,
            args.random_seed,
            avg_basket_value,
            args.sms_cost,
        ),
    ]

    scored_validation = outer_valid[["client_id", "treatment_flg", "target"]].
        copy()
    scored_test = bundle.test[["client_id"]].copy()

    # Persist both raw scores and ranks so the saved artifacts support audit,
    # reporting, and downstream targeting simulations.
    metric_rows: list[dict[str, object]] = []
    qini_frames: list[pd.DataFrame] = []
    curve_frames: list[pd.DataFrame] = []
    params_payload = {
        "run_config": {
            "data_dir": str(args.data_dir.resolve()),
            "output_dir": str(args.output_dir.resolve()),
            "random_seed": args.random_seed,
            "sms_cost": args.sms_cost,
            "avg_basket_value": avg_basket_value,
            "reuse_features": args.reuse_features,
            "skip_sensitivity_sweep": args.skip_sensitivity_sweep,
        },
        "feature_checks": feature_checks,
        "models": {},
    }

    for result in model_results:
        score_column = result.model_name.lower().replace(" ", "_").replace("-",
            "").replace("__", "_") + "_score"
        rank_column = score_column.replace("_score", "_rank")

        scored_validation[score_column] = result.validation_scores
        scored_validation[rank_column] = (
            scored_validation[score_column].rank(ascending=False, method="first
                ").astype(int)
        )
```

```python

        scored_test[score_column] = result.test_scores
        scored_test[rank_column] = scored_test[score_column].rank(ascending=False, method="first").astype(int)

        evaluation = evaluate_ranking(
            y_true=outer_valid["target"].to_numpy(dtype=int),
            treatment=outer_valid["treatment_flg"].to_numpy(dtype=int),
            scores=result.validation_scores,
            avg_basket_value=avg_basket_value,
            sms_cost=args.sms_cost,
        )

        summary_values = dict(evaluation.summary)
        if result.score_kind != "propensity":
            summary_values["predictive_mp"] = np.nan
            summary_values["predictive_optimal_target_share"] = np.nan

        metric_rows.append(
            {
                "model_name": result.model_name,
                "score_kind": result.score_kind,
                "tuning_metric": result.tuning_metric,
                **summary_values,
            }
        )

        qini_frames.append(evaluation.qini_curve.assign(model_name=result.model_name))
        curve_frames.append(evaluation.grid_curve.assign(model_name=result.model_name))

        params_payload["models"][result.model_name] = {
            "score_kind": result.score_kind,
            "best_params": _to_builtin(result.best_params),
            "tuning_metric": float(result.tuning_metric),
        }

    summary_metrics = pd.DataFrame(metric_rows).sort_values("mpu", ascending=False).reset_index(drop=True)
    qini_curves = pd.concat(qini_frames, ignore_index=True)
    model_curves = pd.concat(curve_frames, ignore_index=True)

    sensitivity_df = pd.DataFrame()
    if not args.skip_sensitivity_sweep:
        sensitivity_df = _build_sensitivity_table(
            model_curves,
            default_basket_value=avg_basket_value,
            default_sms_cost=args.sms_cost,
        )

    scored_validation_path = args.output_dir / "phase2_scored_validation.csv"
    scored_test_path = args.output_dir / "phase2_scored_test.csv"
    summary_metrics_path = args.output_dir / "phase2_summary_metrics.csv"
    qini_curve_path = args.output_dir / "phase2_qini_curves.csv"
    liftup_curve_path = args.output_dir / "phase2_liftup_curves.csv"
    sensitivity_path = args.output_dir / "phase2_profit_sensitivity.csv"
    params_path = args.output_dir / "phase2_model_params.json"

    scored_validation.to_csv(scored_validation_path, index=False)
    scored_test.to_csv(scored_test_path, index=False)
    summary_metrics.to_csv(summary_metrics_path, index=False)
    qini_curves.to_csv(qini_curve_path, index=False)
```

```python
    model_curves.to_csv(liftup_curve_path, index=False)
    if not sensitivity_df.empty:
        sensitivity_df.to_csv(sensitivity_path, index=False)
    params_path.write_text(json.dumps(_to_builtin(params_payload), indent=2), encoding="utf-8")

    _save_plot_qini(qini_curves, args.output_dir / "qini_curves.png")
    _save_plot_liftup(model_curves, args.output_dir / "liftup_curves.png")
    _save_plot_profit(model_curves, args.output_dir / "profit_vs_target_depth.png")
    if not sensitivity_df.empty:
        _save_plot_sensitivity(
            sensitivity_df,
            default_basket_value=avg_basket_value,
            default_sms_cost=args.sms_cost,
            output_path=args.output_dir / "profit_sensitivity.png",
        )

    print("Phase II pipeline completed")
    print("==========================")
    print(f"Validation rows: {len(scored_validation):,}")
    print(f"Test rows:       {len(scored_test):,}")
    print(f"Feature count:   {len(bundle.feature_columns):,}")
    print(f"Avg basket:      {avg_basket_value:,.3f}")
    print(f"SMS cost:        {args.sms_cost:,.3f}")
    print()
    print(summary_metrics[["model_name", "qini_auc", "mpu", "predictive_mp"]].to_string(index=False))


if __name__ == "__main__":
    main()
```

Listing 4: Annotated orchestration flow in `phase2_pipeline.py`

B Reproduction Commands

```cd "D:\Business Analytics\retailhero-uplift"
python -m pip install -r .\requirements.txt
python .\verify_phase1_findings.py
python .\phase2_pipeline.py --reuse-features
python .\verify_phase2_outputs.py
```

Listing 5: Commands used to reproduce the work

References

[1] Devriendt, F., Guns, T., and Verbeke, W. (2021). Learning to rank for uplift modeling. In the value-driven uplift modeling literature adapted here for campaign targeting.

[2] Lo, V. S. Y. (2002). The True Lift Model. A treatment-interaction approach to uplift-style modeling.